

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Инженерия Искусственного Интеллекта

Домашняя работа № 5

Дисциплина: Фронтенд разработка

“Основы npm и запуск компонентного frontend-проекта”

Выполнил:

Фиолетов Эдуард Анджеевич

Группа J3213

Проверил:

Добряков Д. Ю.

Санкт-Петербург

2026 г.

Содержание

1. Цель работы	3
2. Задание	3
3. Обоснование выбранного стека	3
4. Используемые файлы проекта	3
5. Структура прт-проекта	3
6. прт-скрипты	4
7. Зависимости проекта	4
8. Конфигурация Vite	5
9. Точка входа	5
10. Компоненты приложения	5
11. Пример компонента	6
12. Проверка результата	6
13. Вывод	7

1. Цель работы

Цель домашней работы - изучить основы работы с npm, структуру frontend-проекта, установку зависимостей, запуск dev-сервера, сборку приложения и создание компонентов.

Работа выполнена на клиентской части Marketplace Alarm Bot 2. Это SPA-интерфейс на React, TypeScript и Vite, который использует npm для управления зависимостями и скриптами разработки.

2. Задание

По заданию необходимо:

- изучить основные команды npm;
- научиться запускать frontend-проект;
- разобраться со структурой проекта;
- подготовить компоненты приложения;
- показать код и результат работы компонентов.

В Marketplace Alarm Bot 2 эти требования покрываются через `package.json`, Vite, React-компоненты, TypeScript, UI-библиотеку и набор npm-скриптов.

3. Обоснование выбранного стека

Marketplace Alarm Bot 2 оформлен как npm-проект в каталоге frontend. Через npm выполняются установка зависимостей, запуск dev-сервера, сборка production-версии, запуск тестов и проверка кода. React выбран как основной frontend-фреймворк проекта, а Vite используется как dev-сервер и сборщик.

Работа выполнена на рабочем frontend-проекте, где npm используется для полного цикла разработки.

4. Используемые файлы проекта

- `./web/frontend/package.json` - npm-скрипты и зависимости
- `./web/frontend/vite.config.ts` - конфигурация Vite
- `./web/frontend/src/main.tsx` - точка входа frontend-приложения
- `./web/frontend/src/App.tsx` - корневой компонент и роутинг
- `./web/frontend/src/pages` - страницы приложения
- `./web/frontend/src/components` - переиспользуемые компоненты

5. Структура npm-проекта

Корень frontend-проекта находится в:

`./web/frontend`

Основные файлы:

- `package.json` - имя проекта, зависимости и npm-скрипты;
- `package-lock.json` - зафиксированные версии зависимостей;

- vite.config.ts - конфигурация Vite;
- tsconfig.json - настройки TypeScript;
- src/main.tsx - точка входа;
- src/App.tsx - корневой компонент;
- src/pages - страницы приложения;
- src/components - компоненты;
- src/api - работа с API;
- src/context - глобальные контексты;
- src/lib - вспомогательные функции.

6. npm-скрипты

В package.json определены скрипты:

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage"
  }
}
```

Назначение команд:

- npm run dev - запуск локального dev-сервера;
- npm run build - проверка TypeScript и сборка production-версии;
- npm run preview - просмотр собранного приложения;
- npm run test - запуск тестов;
- npm run test:watch - запуск тестов в watch-режиме;
- npm run test:coverage - запуск тестов с покрытием.

7. Зависимости проекта

Основные runtime-зависимости:

- react и react-dom - UI-фреймворк и рендеринг;
- react-router-dom - клиентская маршрутизация;
- @tanstack/react-query - работа с серверным состоянием;
- @base-ui/react - headless-компоненты;
- lucide-react - SVG-иконки;
- tailwindcss - utility-first CSS;
- leaflet - карта заказов;
- @dnd-kit/core - drag-and-drop на странице доставки.

Основные dev-зависимости:

- vite - dev-сервер и сборщик;

- typescript - типизация;
- vitest - тесты;
- @testing-library/react - тестирование React-компонентов;
- @vitejs/plugin-react - React-плагин для Vite;
- @tailwindcss/vite - подключение Tailwind к Vite.

8. Конфигурация Vite

Файл vite.config.ts задает плагины, alias, dev-сервер и проху на backend:

```
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': 'http://localhost:8080',
    },
  },
  build: {
    outDir: 'dist',
    emptyOutDir: true,
  },
})
```

В разработке проху направляет запросы /api на Go-сервер, поэтому CORS вручную настраивать не нужно. В production собранный dist обслуживается Go-сервером.

9. Точка входа

Точка входа src/main.tsx создает React-приложение, подключает провайдеры и монтирует компонент в DOM. В приложении используются провайдеры роутера, React Query, авторизации и темы.

Общая схема:

```
createRoot(document.getElementById('root')!).render(
  <BrowserRouter>
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <App />
      </AuthProvider>
    </QueryClientProvider>
  </BrowserRouter>
);
```

10. Компоненты приложения

Компонентная структура делит приложение на страницы и переиспользуемые блоки.

Страницы:

- LoginPage - вход;
- IndexPage - главная сводка;
- OrdersPage - заказы маркетплейса;
- DeliveryPage - доставка по городу;
- ProductsPage - товары и комплекты;
- WarehousesPage - склады;
- HealthPage - состояние сервиса.

Переиспользуемые компоненты:

- AppLayout - общий каркас;
- CommandPalette - поиск и навигация;
- OrdersTable - таблица заказов;
- OrdersMap - карта заказов;
- DeliveryCardCompact, DraggableCard, Timeline, DroppablePanel - элементы доставки;
- components/ui/* - базовые кнопки, таблицы, поля, диалоги и меню.

11. Пример компонента

Страница заказов показывает типичный компонентный подход: страница берет параметр маршрута, загружает данные через хук и передает результат в таблицу.

```
export default function OrdersPage() {
  const { marketplace = 'wb' } = useParams();
  const { data, isLoading } = useOrders(marketplace);

  if (isLoading || !data) {
    return <div className="text-center py-10 text-muted-foreground">...</div>;
  }

  return <OrdersTable orders={data.orders} marketplace={marketplace} />;
}
```

Страница отвечает за маршрут и загрузку данных, а таблица - за отображение заказов.

12. Проверка результата

Проверка выполнялась по npm-командам и пользовательским сценариям:

1. Проверить package.json и список зависимостей.
2. Установить зависимости через `npm install`.
3. Запустить dev-сервер командой `npm run dev`.
4. Проверить работу проху `/api`.
5. Собрать production-версию командой `npm run build`.
6. Запустить тесты командой `npm run test`.
7. Открыть страницы приложения и проверить работу компонентов.

13. Вывод

В работе разобрана структура npm-проекта Marketplace Alarm Bot 2: npm-скрипты, зависимости, конфигурация Vite, точка входа и компонентная структура приложения. Проект использует npm для полного цикла frontend-разработки: запуска, сборки, тестирования и сопровождения компонентного SPA.